AP React – Partie 2
Affichage

Introduction

On rappelle le contexte des notes de frais GSB.

Maintenant que le visiteur peut s'identifier, l'application va lui permettre de gérer les frais liés à ses déplacements professionnels.

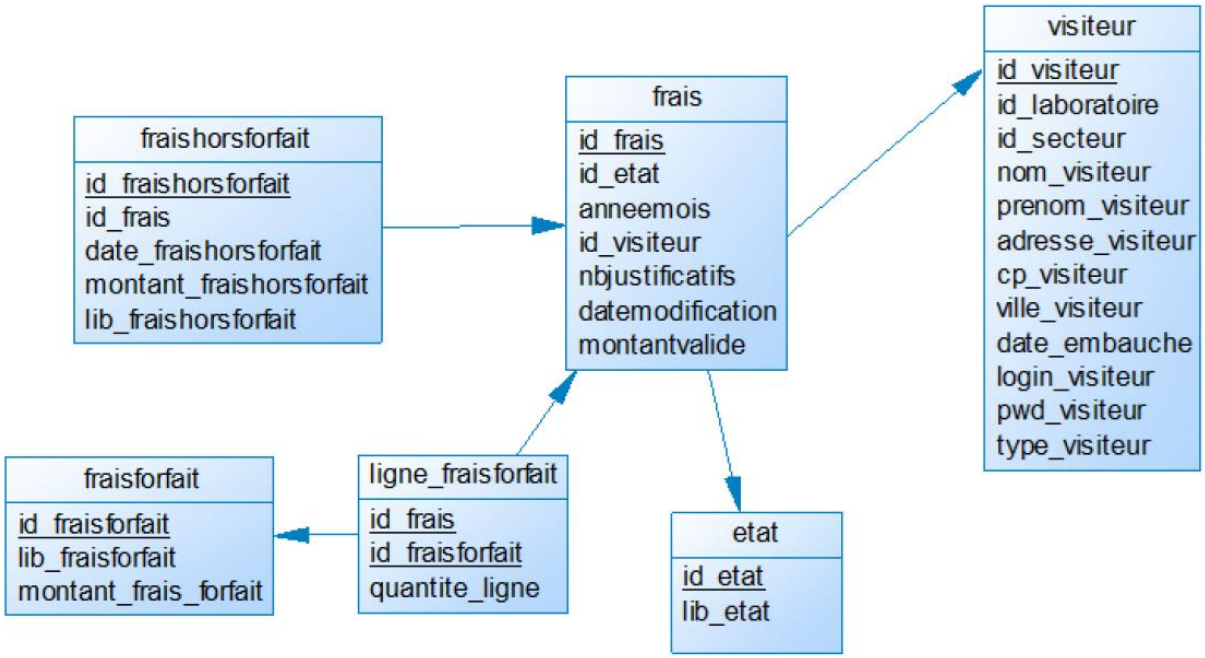
Une note de frais décrit un déplacement, saisi pour un mois précis.

Il peut y avoir plusieurs notes le même mois, pour plusieurs déplacements.

Sur chaque note, les visiteurs ont droit à deux types de frais : les frais au forfait, dont le montant est fixé à l'avance pour lequel on ne saisit qu'une quantité, et les frais hors forfait, pour lesquels il faut saisir la date, le montant et le libellé.

Lorsque le visiteur a saisi une note, il doit la clôturer. Puis, le comptable va valider un montant en fonction des justificatifs fournis. Le visiteur sera remboursé le mois suivant des frais engagés.

On rappelle ci-dessous le modèle de base de données :



Nous allons afficher les notes de frais. Dans un premier temps, nous utiliserons un fichier .json pour stocker les données 'en dur'. Dans les séances suivantes, nous utiliserons une API pour extraire les données de la base et les récupérer au format json.

La structure du code sera la suivante :

```
/src
  /components
    - FraisTable.jsx
  /pages
    - Dashboard.jsx
 /style
    - FraisTable.css
  /data
    - frais.json  # Fichier JSON statique
```

Étape 1 : Créer le fichier JSON statique

Créez un fichier frais.json dans /src/data avec des données simulées :

```
[
  {
    "id_frais": 1,
    "id_etat": 1,
    "anneemois": "2023-10",
    "id_visiteur": 101,
    "nbjustificatifs": 5,
    "datemodification": "2023-10-15",
    "montantvalide": null
  },
  {
    "id_frais": 2,
    "id_etat": 2,
    "anneemois": "2023-11",
    "id_visiteur": 102,
    "nbjustificatifs": 3,
    "datemodification": "2023-11-20",
    "montantvalide": 200.50
  },
  {
    "id_frais": 3,
    "id_etat": 1,
    "anneemois": "2023-09",
    "id_visiteur": 103,
    "nbjustificatifs": 2,
    "datemodification": "2023-09-10",
    "montantvalide": null
  },
  {
    "id_frais": 4,
    "id_etat": 3,
    "anneemois": "2023-12",
    "id_visiteur": 101,
    "nbjustificatifs": 4,
    "datemodification": "2023-12-05",
    "montantvalide": 180.00
  }
]
```

Étape 2 : Créer le composant FraisTable et l'afficher

1. **Copiez le fichier FraisTable.jsx** fourni dans /src/components et
2. **Importez les dépendances nécessaires** :
   - React, useState, et useEffect depuis react.
   - Les données statiques depuis ../data/frais.json : import fraisData from '../data/frais.json';
   - Le fichier CSS pour le style (à créer ensuite).
3. **Déclarez un composant fonctionnel FraisTable.**

Ce composant va gérer l'état fraisList : un tableau pour stocker les données de la table frais.

4. **Déclarez l'état avec useState** en l'initialisant avec un tableau vide.
5. **Complétez le tableau HTML** affiché avec les colonnes suivantes :
   - ID
   - ID État
   - Année-Mois
   - ID Visiteur
   - Nombre de justificatifs
   - Date de modification
   - Montant saisi
   - Montant validé
6. **Utilisez la méthode map** pour parcourir le tableau frais et afficher chaque ligne du tableau.

   Rappel de la syntaxe de map :

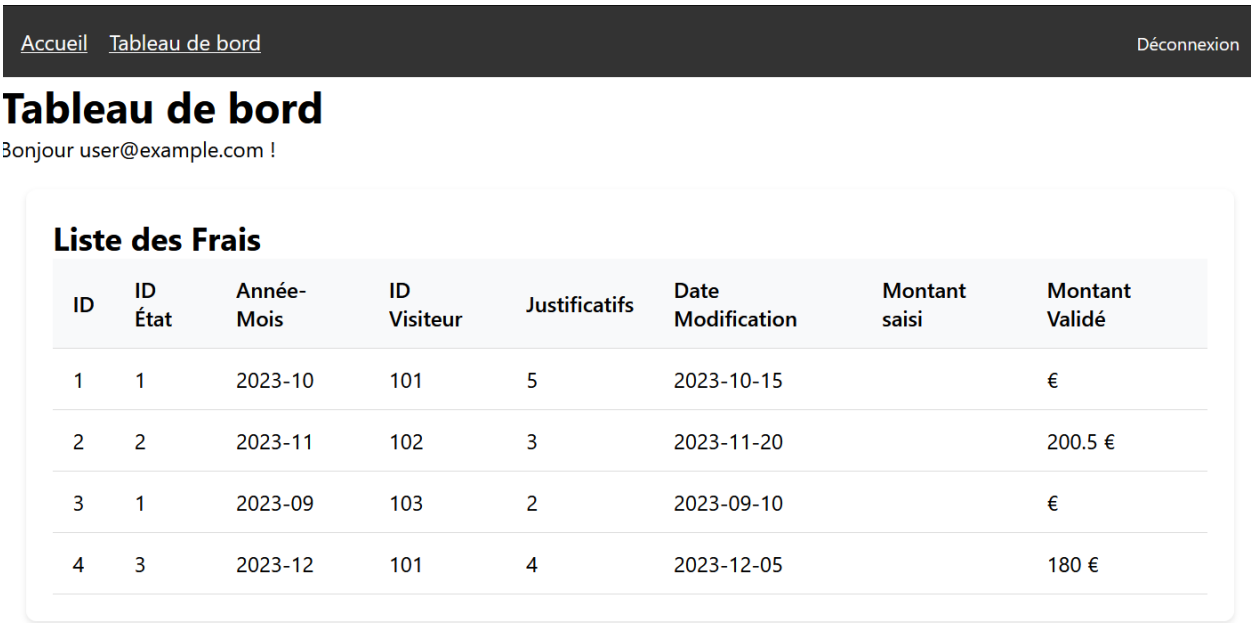
```
tableau.map((element, index) => (
  <tr key={element.id}>
    <td>{element.propriete1}</td>
    <td>{element.propriete2}</td>
  </tr>
))
```

   Dans notre cas, le tableau correspond à l'état fraisList et l'élément correspond à une ligne du tableau, c'est-à-dire un frais. Pour chaque frais, nous affichons les propriétés ID, ID état, Année-mois etc.

   Attention :
   - il faut bien respecter le nom des colonnes de la table pour nommer les propriétés affichées.
   - On rappelle que le montant saisi n'est pas dans la table frais, il est calculé comme la somme des montants des frais au forfait et hors forfait. Pour le moment, on laisse ce champ vide dans le tableau affiché.

Remarque : en React, on utilise map pour itérer sur les listes pour plusieurs raisons, et notamment car la syntaxe est plus simple. Vous trouverez en annexe l'équivalent de ce code avec une boucle for.

7. **Ajoutez l'ID du frais comme clé unique (key)** à chaque ligne du tableau pour éviter les avertissements React.
8. Afficher le composant FraisTable dans le tableau de bord en l'ajoutant dans Dashboard.jsx. Vérifiez que vous obtenez l'affichage suivant :



ID	ID État	Année-Mois	ID Visiteur	Justificatifs	Date Modification	Montant saisi	Montant Validé
1	1	2023-10	101	5	2023-10-15		€
2	2	2023-11	102	3	2023-11-20		200.5 €
3	1	2023-09	103	2	2023-09-10		€
4	3	2023-12	101	4	2023-12-05		180 €

Etape 3 : Simuler le chargement des données depuis un serveur

Afin d'imiter un appel API, nous allons ajouter la simulation du chargement des données, en affichant « Chargement des frais… » pendant 500 ms avant d'afficher la liste des frais.

En effet, dans une application réelle, les données sont souvent récupérées depuis une **API** ou une **base de données**, ce qui prend du temps (à cause de la latence réseau, du traitement serveur, etc.). Pendant ce temps, il est important d'afficher un **état de chargement** (ex: un spinner, un message "Chargement...") pour informer l'utilisateur que l'application est en train de travailler.

En utilisant un fichier JSON statique, **les données sont disponibles instantanément**, ce qui ne reflète pas la réalité.

L'utilisation de **useEffect** pour simuler un chargement dans une application React est une technique courante pour **reproduire le comportement d'un appel API asynchrone** (comme une requête vers un backend). Cela permet de tester l'interface utilisateur dans des conditions réalistes, où les données mettent un certain temps à se charger.

useEffect est un **hook React** (une fonction fournie par React) qui permet d'effectuer des **effets de bord** dans un composant fonctionnel.

Rappel de la syntaxe de useEffect() :

```
useEffect(() => {
  // Code à exécuter après le rendu du composant
  return () => {
```

```
    // Code de nettoyage (ex: annuler un abonnement)
  };
}, [dependencies]); // Tableau de dépendances
```

1. Dans le composant FraisTable, déclarez un état loading initialisé à true.
2. Insérez le code suivant au bon endroit dans le composant :

```
useEffect(() => {
  // Simulation d'un appel API avec un délai de 500 ms
  setTimeout(() => {
    setFraisList(fraisData); // Met à jour l'état avec les données du fichier JSON
    setLoading(false); // Met fin à l'état de chargement
  }, 50000); // Délai pour simuler un chargement
}, []); // Tableau de dépendances vide = exécute une seule fois

if (loading) return <div><b>Chargement des frais...</b></div>;
```

setTimeout : Simule un délai de 500 ms (comme un appel API lent).

- Après 500 ms, les données (fraisData) sont assignées à l'état frais via setFrais.
- L'état loading est mis à false pour indiquer que le chargement est terminé.

La dernière ligne ajoute un affichage conditionnel : tant que `loading` est `true`, un message de chargement est affiché. Une fois `loading` passé à `false`, le tableau des frais est affiché.

3. Pour vérifier que cela fonctionne, revenez à l'accueil puis cliquez sur Tableau de bord et vérifiez que vous avez l'affichage suivant avant de voir le tableau apparaître :

Etape 4 : Filtrer l'affichage

L'intérêt d'utiliser React est d'obtenir une application dynamique. Nous allons ajouter une fonctionnalité pour pouvoir filtrer les frais affichés en recherchant une date, un montant ou un ID visiteur. Le filtrage ne nécessitera pas de rechargement de la page, comme cela aurait été le cas dans votre application Laravel. La mise à jour se fera dynamiquement, directement dans le navigateur sans interaction avec la source des données.

1. Dans FraisTable.jsx, déclarer un état searchTerm, initialisé avec une chaîne de caractères vide. Comme d'habitude, on déclare en même temps la fonction associée setSearchTerm qui permettra de mettre à jour searchTerm quand l'utilisateur tape dans le champ.

2. Ajouter le code suivant avant l'affichage (avant le return) pour obtenir la liste des frais dont le terme entré dans la barre de recherche est une partie de anneemois, ou de id_visiteur.

```
// Logique de filtrage : filtre les frais en fonction du terme de recherche
const filteredFrais = fraisList.filter((frais) =>
  frais.anneemois.includes(searchTerm) ||
  frais.id_visiteur.toString().includes(searchTerm)
);
```

3. Ajouter le champ de recherche juste au-dessus du tableau avec le code suivant. A chaque changement du texte saisi dans le champ, l'état searchTerm sera mis à jour, grâce à la ligne onChange={(e) => setSearchTerm(e.target.value)}

```
{/* Champ de recherche pour le filtrage */}
<div className="search-container">
  <input
    type="text"
    placeholder="Rechercher par année-mois, ID visiteur ou montant..."
    value={searchTerm}
    onChange={(e) => setSearchTerm(e.target.value)} // Met à jour searchTerm
  />
</div>
```

4. Modifier l'affichage des frais pour afficher la liste filteredFrais au lieu de la liste fraisList.
5. Vous avez certainement remarqué que nous affichons de la même façon les notes frais, que le montant ait été validé ou non.
   On peut chaîner les filtres en enchaînant les appels à la méthode filter.
   D'abord, on exclut les frais avec montantvalide=null. Ensuite, on applique le filtre de recherche (searchTerm). Cela donne le code suivant :

```
// Logique de filtrage : filtre les frais en fonction du terme de recherche
const filteredFrais = fraisList
.filter((frais) => frais.montantvalide !== null) // Exclut les frais avec montantvalide = null
.filter((frais) =>
  frais.anneemois.includes(searchTerm) ||
  frais.id_visiteur.toString().includes(searchTerm)
);
```

   Modifiez votre code. Regardez l'affichage. Vous ne devriez avoir que deux frais affichés.
6. Avec ce filtre, nous n'avons plus du tout accès aux notes de frais non validées. C'est dommage, nous aimerions que l'utilisateur puisse décider de l'afficher ou non. Nous allons donc ajouter une case à cocher et gérer l'affichage en fonction de son état.
   - Déclarer un état filterNonNull initalisé à true
   - Modifier le premier filtre pour ne l'appliquer sur si la case est cochée :

```
.filter((f) => !filterNonNull || f.montantvalide !== null) // Applique le filtre seulement si coché
```

   - Dans le JSX, ajouter la case à cocher au-dessus de la barre de recherche :

```
{/* Case à cocher */}
<div className="filter-container">
  <label>
    <input
      type="checkbox"
      checked={filterNonNull}
      onChange={(e) => setFilterNonNull(e.target.checked)}
    />
    Afficher seulement les frais avec un montant validé
  </label>
</div>
```

7. Ajouter le code suivant dans le css pour améliorer le style de la case à cocher et du champ de recherche.

```
.search-container {
  margin-bottom: 20px;
}

.search-container input {
  padding: 10px;
  width: 100%;
  border: 1px solid #ddd;
  border-radius: 4px;
  font-size: 14px;
}

.filter-container {
  margin-bottom: 15px;
}

.filter-container label {
  display: flex;
  align-items: center;
  gap: 8px;
  cursor: pointer;
}
```

8. Ajouter un filtre pour n'afficher que les notes de frais dont le montant validé est supérieur à une valeur saisie par l'utilisateur dans un champ. Par exemple, on pourra obtenir l'affichage suivant si l'utilisateur saisit 200 :

Annexes

Annexe 1 : Equivalent de l'itération sur la liste des frais avec for à la place de map

```
const rows = [];
for (let i = 0; i < fraisList.length; i++) {
  const frais = fraisList[i];
  rows.push(
    <tr key={frais.id_frais}>
      <td>{frais.id_frais}</td>
      <td>{frais.anneemois}</td>
    </tr>
  );
}
return <>{rows}</>;
```